Terminal Buddy: Technical Build Specification (V3.0 - Deep System Integration)

Lead Architect: Fred **Core Language:** Rust 1.80+ (Stable) **Specialty:** Kernel-level Observation & Correlation

1. The Correlation Engine (The "Why")

The most critical missing piece is the **Correlation Engine**. This links a specific `CommandID` to three distinct data streams.

The Correlation Triple:

1. **Stream A (Terminal):** Raw StdOut/StdErr (The "Result").
2. **Stream B (Filesystem/Process):** The Delta in files and PIDs (The "Impact").
3. **Stream C (System Logs):** Kernel/OS logs filtered by timestamp (The "Root Cause").

Correlation Logic (Rust):

```
pub struct EventCorrelator {
    pub command_id: uuid::Uuid,
    pub timestamp_range: (DateTime<Utc>, DateTime<Utc>),
}

impl EventCorrelator {
    // Queries the OS logs for events that happened DURING command execution
    pub fn fetch_system_logs(&self) -> Vec<OSLogEntry> {
        #[cfg(target_os = "linux")]
        let logs = query_journalctl(self.timestamp_range);

        #[cfg(target_os = "macos")]
        let logs = query_unified_logging(self.timestamp_range);

        logs
    }
}
```

2. The Hardware Abstraction Layer (HAL)

To make this "Provable Tech," we use specific Rust crates to tap into the computer's "Nervous System."

Component	Source of Truth	Rust Implementation
IO Monitoring	`/proc/diskstats` (Linux)	`iotop` logic via `sysinfo` / `procfs`
System Logs	`journalctl` / `syslog`	`libsystemd` bindings
File Watcher	Kernel `inotify` / `FSEvents`	`notify` crate (recursive, async)
Network	`netstat` / `/proc/net`	`net-stat` crate

3. Detailed API & IPC Schema (Bincode/RPC)

The **Sidecar Daemon** exposes a JSON-RPC interface for the **Canvas Dashboard** and a high-speed Bincode interface for the **GUI**.

Endpoint: `GetTeachableMoment(command_id)`

Request: UUID of the command. **Response (The "Hard Design"):**

```
{
  "command": "docker-compose up",
  "terminal_summary": "Error: Port 8080 already in use",
  "system_impact": {
    "files_modified": ["./docker-compose.yml"],
    "new_processes": []
  },
  "root_cause_logs": [
    {
      "source": "kernel",
      "level": "error",
      "message": "TCP: [::]:8080 bound by process 1290 (nginx)"
    }
  ],
  "buddy_explanation": "You can't start this because Nginx is already holding por
}
```

4. The Memvid Frame Structure (The "Hard Space")

Each frame in the `.mv2` file is an atomic "Reality Snapshot."

```
pub struct MemvidFrame {
    pub frame_type: u8,         // 0: Text, 1: SystemSnap, 2: Correlation
```

```
    pub data: Vec<u8>,           // Compressed Zstd payload
    pub vector_embedding: [f32; 1536], // For semantic history search
}
```

- **Linearity:** Frames are appended in real-time.
- **Scrubbing:** When you scrub the timeline in **Canvas**, the UI reconstructs the system state by replaying these frames.

5. Security & Privilege Model

Since we are reading system logs, the **Sidecar** must run with specific capabilities.

- **Linux:** `CAP_DAC_READ_SEARCH` (for logs) and `CAP_SYS_PTRACE` (for process monitoring).
- **User Control:** Terminal Buddy will prompt for sudo *once* to spawn the sidecar in "Observation Mode."

6. Build Directives (Phase 1 Deep Dive)

1. `src/daemon/log_pipe.rs`: Create an asynchronous stream that parses `journalctl -f` into structured Rust enums.
2. `src/daemon/state_diff.rs`: Implement the memory-mapped storage for "Pre-Execution" system states.
3. `src/correlator/mod.rs`: The brain that matches the Shell PID to the system log timestamps.